FORMATION REACT.JS · DÉMO 004-B

Règles des hooks : les changements

On voit les règles une par une, avec les exemples de `src/components/BadExample.jsx`. Ensuite, on détecte ces erreurs en local avec ESLint, puis on les bloque automatiquement avec Husky (avant le commit ou le push), et enfin dans la pipeline.

La règle d'or : **on appelle les hooks au premier niveau du composant**, toujours dans le même ordre à chaque render.

Étape 1 · Règle 1 : pas de hook dans une condition

Le code interdit :

```
// Hook dans un if (condition)
export default function Profile({ logged }) {
  if (logged) {
    const [name, setName] = useState("Hicham"); // INTERDIT !
  }

  return <div>Profile</div>;
}
```

Pourquoi ? React garde une liste ordonnée des hooks :

- Premier render (logged = true) : Hook #1 = useState("Hicham")
- Deuxième render (logged = false) : Hook #1 = ??? (absent !)

L'ordre des hooks a changé → crash ou bugs bizarres.

L'erreur ESLint :

```
7:29  error  React Hook "useState" is called conditionally. React Hooks must be called in the
exact same order in every component render  react-hooks/rules-of-hooks
```

La bonne version : le hook est toujours appelé, c'est l'affichage qui dépend de la condition.

```
export default function Profile({ logged }) {
  const [name] = useState("Hicham"); // toujours appelé, au début du composant

  if (!logged) return <div>Profile</div>;
  return <div>Profile de {name}</div>;
}
```

Étape 2 · Règle 2 : pas de hook dans une boucle

Le code interdit :

```
// Hook dans une boucle (for, map)
function List({ items }) {
  items.map((x) => {
    const [selected, setSelected] = useState(false); // INTERDIT !
    return <div>{x}</div>;
  });

  return null;
}
```

Pourquoi ?

- Si items = [A, B, C] → 3 hooks créés
- Si items = [A, B] → 2 hooks maintenant

React : « Mais où est passé le 3ème hook ?? » → crash. Le nombre de hooks ne doit jamais changer entre les renders.

L'erreur ESLint :

```
25:37  error  React Hook "useState" cannot be called inside a callback. React Hooks must be
called in a React function component or a custom React Hook function  react-hooks/rules-of-hooks
```

La bonne version : un petit composant par élément, chacun avec son propre state.

```
function Item({ x }) {
  const [selected, setSelected] = useState(false);
  return <div onClick={() => setSelected(!selected)}>{x}</div>;
}

function List({ items }) {
  return items.map((x) => <Item key={x} x={x} />);
}
```

Étape 3 · Règle 3 : seulement dans un composant ou un custom hook

Le code interdit :

```
// Hook dans une fonction JS classique
function doSomething() {
  const [x, setX] = useState(0); // INTERDIT !
}
```

Pourquoi ? doSomething n'est ni :

- un composant React (il faudrait DoSomething, avec une majuscule)
- un custom hook (il faudrait useDoSomething)

Les hooks ne marchent que pendant le render d'un composant. Si doSomething est appelée ailleurs (au clic, dans un setTimeout...) → erreur « Invalid hook call ». Et avec ce nom, ESLint signale l'erreur : il repère les hooks grâce à la majuscule et au « use ».

L'erreur ESLint :

```
43:21  error  React Hook "useState" is called in function "doSomething" that is neither a React
function component nor a custom React Hook function. React component names must start with an
uppercase letter. React Hook names must start with the word "use"  react-hooks/rules-of-hooks
```

La bonne version : on en fait un custom hook (nom en `use…`), appelé au début d'un composant.

```
function useSomething() {
  const [x, setX] = useState(0);
  return { x, setX };
}

function MyComponent() {
  const { x } = useSomething();
  return <div>{x}</div>;
}
```

Étape 4 · Règle 4 : pas de hook dans une fonction imbriquée

Le code interdit :

```
// Hook dans une fonction imbriquée (handler, callback)
function Counter() {
  const handleClick = () => {
    const [count, setCount] = useState(0); // INTERDIT !
  };

  return <button onClick={handleClick}>+1</button>;
}
```

Pourquoi ? handleClick est appelée au clic, pas pendant le render du composant. Au clic, React plante → erreur « Invalid hook call ».

L'erreur ESLint :

```
60:31  error  React Hook "useState" is called in function "handleClick" that is neither a React
function component nor a custom React Hook function. React component names must start with an
uppercase letter. React Hook names must start with the word "use"  react-hooks/rules-of-hooks
```

La bonne version : le hook est au début du composant, et le handler l'utilise.

```
function Counter() {
  const [count, setCount] = useState(0);
  const handleClick = () => setCount(count + 1);

  return <button onClick={handleClick}>+1 ({count})</button>;
}
```

Étape 5 · Détecter les erreurs en local avec ESLint

Le projet Vite a déjà ESLint, avec le plugin des hooks (`eslint-plugin-react-hooks`) dans `eslint.config.js` :

```
import reactHooks from "eslint-plugin-react-hooks";

export default defineConfig([
  {
    files: ["**/*.{js,jsx}"],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended, // les règles des hooks
      reactRefresh.configs.vite,
    ],
  },
]);
```

Et le script `lint` dans `package.json` :

```
"scripts": {
  "lint": "eslint ."
}
```

On lance :

```
npm install
npm run lint
```

Résultat : les 4 hooks mal placés sont signalés (`react-hooks/rules-of-hooks`), avec la ligne et la colonne. Il y a aussi des erreurs `no-unused-vars` (les variables du state ne sont pas utilisées dans les exemples).

```
✖ 13 problems (13 errors, 0 warnings)
```

Dans VS Code, avec l'extension ESLint, les erreurs sont soulignées en rouge directement dans le fichier.

Étape 6 · Automatiser avec Husky (pre-commit, pre-push)

Problème : on peut oublier de lancer `npm run lint` . Husky le lance tout seul, avant chaque `git commit` (ou `git push`). S'il y a une erreur, l'action est bloquée. Tout se passe en local.

Installer Husky (déjà fait dans ce projet) :

```
npm install -D husky
npx husky init   # crée .husky/pre-commit et ajoute "prepare": "husky" dans package.json
```

Dans `.husky/pre-commit` , on remplace `npm test` par `npm run lint` :

```
# Husky : avant chaque git commit, on lance ESLint. S'il y a une erreur, le commit est bloqué.
npm run lint
```

Si le fichier n'est pas exécutable (`permission denied`), on ajoute la permission :

```
chmod +x .husky/pre-commit
```

Pour le montrer :

```
git init
npm install          # le script prepare active Husky dans ce dépôt git
git add .
git commit -m "test" # ESLint tourne → erreurs → commit bloqué
```

Résultat :

```
✖ 13 problems (13 errors, 0 warnings)
husky - pre-commit script failed (code 1)
```

On corrige un hook dans `BadExample.jsx`, on relance le commit : tant qu'il reste une erreur, il est bloqué.

Variante : bloquer le `git push` au lieu du commit (on peut committer, mais pas envoyer du code avec des erreurs) :

```
echo "npm run lint" > .husky/pre-push
chmod +x .husky/pre-push
```

Étape 7 · Dans la pipeline (CI)

Husky tourne sur la machine du développeur : on peut le contourner (`git commit --no-verify`). Donc on relance le même `npm run lint` dans la pipeline, à chaque push et à chaque pull request. Si le lint échoue, la pipeline est rouge et on ne merge pas.

Exemple avec GitHub Actions, dans `.github/workflows/lint.yml` (à la racine du dépôt) :

```
name: Lint
on: [push, pull_request]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: npm ci
      - run: npm run lint
```

Même idée avec GitLab CI, Jenkins… : une étape qui lance `npm ci` puis `npm run lint`.

Les 3 niveaux :

- **Dans l'éditeur** : l'extension ESLint souligne l'erreur pendant qu'on code
- **En local** : Husky bloque le commit ou le push
- **Dans la pipeline** : le lint bloque le merge, même si quelqu'un a contourné Husky

Documentation

- Les règles des hooks : https://react.dev/reference/rules/rules-of-hooks
- Le plugin ESLint des hooks : https://www.npmjs.com/package/eslint-plugin-react-hooks
- Husky : https://typicode.github.io/husky/get-started.html

- GitHub Actions, Node.js : https://docs.github.com/fr/actions/use-cases-and-examples/building-and-testing/building-and-testing-nodejs